



Student Name : Pranay Mestry

Batch : CS2

Roll Number : CS31

Aim

To implement Principal Component Analysis (PCA) for dimensionality reduction on the Breast Cancer Wisconsin (Diagnostic) dataset, visualize the high-dimensional feature space in a reduced 2D plane, and evaluate its impact on classification performance.

Objectives

1. To understand the concepts of dimensionality reduction, feature correlation, and Principal Component Analysis.
2. To preprocess, clean, and scale high-dimensional real-world data using StandardScaler.
3. To implement PCA using Python to compress feature dimensions from 30 continuous features down to 2 principal components.
4. To calculate and plot the explained variance ratio and cumulative explained variance.
5. To visualize the transformed 2D dataset and analyze class separability.
6. To evaluate and compare classification model performance (e.g., Logistic Regression) before and after applying PCA.

Relevant Theory

Support Vector Machine

Principal Component Analysis (PCA) Principal Component Analysis (PCA) is an unsupervised machine learning technique used for dimensionality reduction. It transforms a dataset containing a large set of correlated features into a

smaller set of uncorrelated variables called Principal Components (\$PCs\$), while preserving as much data variance as possible.

Mathematical Steps:

1. Standardization: Scale each feature to have a mean of 0 ($\mu = 0$) and standard deviation of 1 ($\sigma = 1$).
2. Covariance Matrix: Compute the covariance matrix C to quantify feature correlations:
$$C = \frac{1}{n-1} X^T X$$
3. Eigen decomposition: Compute eigenvalues (λ) and eigenvectors (v) of matrix C :
$$C v = \lambda v$$
 - Eigenvectors represent the direction of the principal axes.
 - Eigenvalues indicate the amount of variance along those axes.
4. Projection: Project original data X onto top k principal component directions W :
$$X_{\text{pca}} = X \cdot W$$

Dataset Information

1. Dataset Name: Breast Cancer Wisconsin (Diagnostic) Dataset
2. Kaggle Link: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>
3. Classes: 2 binary classes (Malignant [M] and Benign [B])
4. Features: 30 continuous features computed from digitized images of fine needle aspirates (FNA) of a breast mass (e.g., radius_mean, texture_mean, perimeter_mean, area_mean, smoothness_mean, concavity_mean, etc.).
5. Non-Feature Columns: id (identifier) and Unnamed: 32 (empty column).

Task 1: Import Libraries and Load Dataset

Import core Python libraries (numpy, pandas, matplotlib, seaborn, sklearn) and load data.csv obtained from Kaggle.

In []: *# Task 1: Import Libraries and Load Dataset*

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset (Upload data.csv to Google Colab first)
from google.colab import files
uploaded = files.upload()

df = pd.read_csv('data.csv')

# Display dataset information
print("Dataset loaded successfully!")
print("\nFirst 5 Rows:")
print(df.head())

print("\nDataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
print(df.describe())
```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving data.csv to data.csv
Dataset loaded successfully!

First 5 Rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
0	842302	M	17.99	10.38	122.80	100
1	842517	M	20.57	17.77	132.90	132
2	84300903	M	19.69	21.25	130.00	120
3	84348301	M	11.42	20.38	77.58	38
4	84358402	M	20.29	14.34	135.10	129

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	0.11840	0.27760	0.3001	0.14710
1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790
3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430

	texture_worst	perimeter_worst	area_worst	smoothness_worst
0	17.33	184.60	2019.0	0.1622
1	23.41	158.80	1956.0	0.1238
2	25.53	152.50	1709.0	0.1444
3	26.50	98.87	567.7	0.2098
4	16.67	152.20	1575.0	0.1374

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst
0	0.6656	0.7119	0.2654	0.46
1	0.1866	0.2416	0.1860	0.27
2	0.4245	0.4504	0.2430	0.36
3	0.8663	0.6869	0.2575	0.66
4	0.2050	0.4000	0.1625	0.23

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

Dataset Shape: (569, 33)

Column Names:

```
['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean', 'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean', 'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se', 'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se', 'fractal_dimension_se', 'radius_worst', 'texture_worst', 'perimeter_worst', 'area_worst', 'smoothness_worst', 'compactness_worst', 'concavity_worst', 'concave points_worst', 'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32']
```

Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64
32	Unnamed: 32	0 non-null	float64

dtypes: float64(31), int64(1), object(1)

memory usage: 146.8+ KB

Missing Values:

id 0

```

diagnosis          0
radius_mean        0
texture_mean        0
perimeter_mean     0
area_mean          0
smoothness_mean    0
compactness_mean    0
concavity_mean     0
concave points_mean 0
symmetry_mean      0
fractal_dimension_mean 0
radius_se          0
texture_se          0
perimeter_se       0
area_se            0
smoothness_se      0
compactness_se     0
concavity_se       0
concave points_se  0
symmetry_se        0
fractal_dimension_se 0
radius_worst       0
texture_worst      0
perimeter_worst    0
area_worst         0
smoothness_worst   0
compactness_worst  0
concavity_worst    0
concave points_worst 0
symmetry_worst     0
fractal_dimension_worst 0
Unnamed: 32        569
dtype: int64

```

Statistical Summary:

	id	radius_mean	texture_mean	perimeter_mean	area_me
an \					
count	5.690000e+02	569.000000	569.000000	569.000000	569.0000
00					
mean	3.037183e+07	14.127292	19.289649	91.969033	654.8891
04					
std	1.250206e+08	3.524049	4.301036	24.298981	351.9141
29					
min	8.670000e+03	6.981000	9.710000	43.790000	143.5000
00					
25%	8.692180e+05	11.700000	16.170000	75.170000	420.3000
00					
50%	9.060240e+05	13.370000	18.840000	86.240000	551.1000
00					
75%	8.813129e+06	15.780000	21.800000	104.100000	782.7000
00					
max	9.113205e+08	28.110000	39.280000	188.500000	2501.0000
00					

smoothness_mean compactness_mean concavity_mean concave point

s_mean \				
count	569.000000	569.000000	569.000000	569.00
0000				
mean	0.096360	0.104341	0.088799	0.04
8919				
std	0.014064	0.052813	0.079720	0.03
8803				
min	0.052630	0.019380	0.000000	0.00
0000				
25%	0.086370	0.064920	0.029560	0.02
0310				
50%	0.095870	0.092630	0.061540	0.03
3500				
75%	0.105300	0.130400	0.130700	0.07
4000				
max	0.163400	0.345400	0.426800	0.20
1200				

	symmetry_mean	...	texture_worst	perimeter_worst	area_worst \
count	569.000000	...	569.000000	569.000000	569.000000
mean	0.181162	...	25.677223	107.261213	880.583128
std	0.027414	...	6.146258	33.602542	569.356993
min	0.106000	...	12.020000	50.410000	185.200000
25%	0.161900	...	21.080000	84.110000	515.300000
50%	0.179200	...	25.410000	97.660000	686.500000
75%	0.195700	...	29.720000	125.400000	1084.000000
max	0.304000	...	49.540000	251.200000	4254.000000

	smoothness_worst	compactness_worst	concavity_worst \
count	569.000000	569.000000	569.000000
mean	0.132369	0.254265	0.272188
std	0.022832	0.157336	0.208624
min	0.071170	0.027290	0.000000
25%	0.116600	0.147200	0.114500
50%	0.131300	0.211900	0.226700
75%	0.146000	0.339100	0.382900
max	0.222600	1.058000	1.252000

	concave points_worst	symmetry_worst	fractal_dimension_worst \
count	569.000000	569.000000	569.000000
mean	0.114606	0.290076	0.083946
std	0.065732	0.061867	0.018061
min	0.000000	0.156500	0.055040
25%	0.064930	0.250400	0.071460
50%	0.099930	0.282200	0.080040
75%	0.161400	0.317900	0.092080
max	0.291000	0.663800	0.207500

	Unnamed: 32
count	0.0
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN

75%	NaN
max	NaN

[8 rows x 32 columns]

Task 2: Explore the Dataset

Inspect missing values, view dataset summary statistics, check column data types, and inspect target class distributions (diagnosis).

```
In [ ]: # Task 2: Explore the Dataset

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv('data.csv')

# 1. Display first five rows
print("First 5 Rows:")
print(df.head())

# 2. Dataset shape
print("\nDataset Shape:", df.shape)

# 3. Check missing values
print("\nMissing Values in Each Column:")
print(df.isnull().sum())

print("\nTotal Missing Values:", df.isnull().sum().sum())

# 4. Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# 5. Check column data types
print("\nColumn Data Types:")
print(df.dtypes)

# 6. Dataset information
print("\nDataset Information:")
df.info()

# 7. Inspect target class distribution
if 'diagnosis' in df.columns:
    print("\nDiagnosis Class Distribution:")
    print(df['diagnosis'].value_counts())

    print("\nDiagnosis Class Distribution (Percentage):")
```

```
print(df['diagnosis'].value_counts(normalize=True) * 100)

# 8. Visualize target class distribution
sns.countplot(data=df, x='diagnosis')
plt.title("Diagnosis Class Distribution")
plt.xlabel("Diagnosis (B = Benign, M = Malignant)")
plt.ylabel("Count")
plt.show()
else:
    print("\nError: 'diagnosis' column not found.")
    print("Available columns:", df.columns.tolist())
```

First 5 Rows:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
0	842302	M	17.99	10.38	122.80	100
1	842517	M	20.57	17.77	132.90	132
2	84300903	M	19.69	21.25	130.00	120
3	84348301	M	11.42	20.38	77.58	38
4	84358402	M	20.29	14.34	135.10	129

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	0.11840	0.27760	0.3001	0.14710
1	0.08474	0.07864	0.0869	0.07017
2	0.10960	0.15990	0.1974	0.12790
3	0.14250	0.28390	0.2414	0.10520
4	0.10030	0.13280	0.1980	0.10430

	texture_worst	perimeter_worst	area_worst	smoothness_worst
0	17.33	184.60	2019.0	0.1622
1	23.41	158.80	1956.0	0.1238
2	25.53	152.50	1709.0	0.1444
3	26.50	98.87	567.7	0.2098
4	16.67	152.20	1575.0	0.1374

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst
0	0.6656	0.7119	0.2654	0.46
1	0.1866	0.2416	0.1860	0.27
2	0.4245	0.4504	0.2430	0.36
3	0.8663	0.6869	0.2575	0.66
4	0.2050	0.4000	0.1625	0.23

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

[5 rows x 33 columns]

Dataset Shape: (569, 33)

Missing Values in Each Column:

id	0
----	---

```

diagnosis          0
radius_mean        0
texture_mean        0
perimeter_mean     0
area_mean          0
smoothness_mean    0
compactness_mean    0
concavity_mean      0
concave points_mean 0
symmetry_mean       0
fractal_dimension_mean 0
radius_se          0
texture_se          0
perimeter_se        0
area_se            0
smoothness_se       0
compactness_se      0
concavity_se        0
concave points_se   0
symmetry_se         0
fractal_dimension_se 0
radius_worst        0
texture_worst        0
perimeter_worst     0
area_worst          0
smoothness_worst    0
compactness_worst    0
concavity_worst     0
concave points_worst 0
symmetry_worst       0
fractal_dimension_worst 0
Unnamed: 32         569
dtype: int64

```

Total Missing Values: 569

Summary Statistics:

	id	radius_mean	texture_mean	perimeter_mean	area_me
an \					
count	5.690000e+02	569.000000	569.000000	569.000000	569.0000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.8891
std	1.250206e+08	3.524049	4.301036	24.298981	351.9141
min	8.670000e+03	6.981000	9.710000	43.790000	143.5000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.3000
50%	9.060240e+05	13.370000	18.840000	86.240000	551.1000
75%	8.813129e+06	15.780000	21.800000	104.100000	782.7000
max	9.113205e+08	28.110000	39.280000	188.500000	2501.0000

	smoothness_mean	compactness_mean	concavity_mean	concave point
s_mean \				
count	569.000000	569.000000	569.000000	569.00
0000				
mean	0.096360	0.104341	0.088799	0.04
8919				
std	0.014064	0.052813	0.079720	0.03
8803				
min	0.052630	0.019380	0.000000	0.00
0000				
25%	0.086370	0.064920	0.029560	0.02
0310				
50%	0.095870	0.092630	0.061540	0.03
3500				
75%	0.105300	0.130400	0.130700	0.07
4000				
max	0.163400	0.345400	0.426800	0.20
1200				

	symmetry_mean	...	texture_worst	perimeter_worst	area_worst	\
count	569.000000	...	569.000000	569.000000	569.000000	
mean	0.181162	...	25.677223	107.261213	880.583128	
std	0.027414	...	6.146258	33.602542	569.356993	
min	0.106000	...	12.020000	50.410000	185.200000	
25%	0.161900	...	21.080000	84.110000	515.300000	
50%	0.179200	...	25.410000	97.660000	686.500000	
75%	0.195700	...	29.720000	125.400000	1084.000000	
max	0.304000	...	49.540000	251.200000	4254.000000	

	smoothness_worst	compactness_worst	concavity_worst	\
count	569.000000	569.000000	569.000000	
mean	0.132369	0.254265	0.272188	
std	0.022832	0.157336	0.208624	
min	0.071170	0.027290	0.000000	
25%	0.116600	0.147200	0.114500	
50%	0.131300	0.211900	0.226700	
75%	0.146000	0.339100	0.382900	
max	0.222600	1.058000	1.252000	

	concave points_worst	symmetry_worst	fractal_dimension_worst	\
count	569.000000	569.000000	569.000000	
mean	0.114606	0.290076	0.083946	
std	0.065732	0.061867	0.018061	
min	0.000000	0.156500	0.055040	
25%	0.064930	0.250400	0.071460	
50%	0.099930	0.282200	0.080040	
75%	0.161400	0.317900	0.092080	
max	0.291000	0.663800	0.207500	

	Unnamed: 32
count	0.0
mean	NaN
std	NaN
min	NaN

25%	NaN
50%	NaN
75%	NaN
max	NaN

[8 rows x 32 columns]

Column Data Types:

id	int64
diagnosis	object
radius_mean	float64
texture_mean	float64
perimeter_mean	float64
area_mean	float64
smoothness_mean	float64
compactness_mean	float64
concavity_mean	float64
concave points_mean	float64
symmetry_mean	float64
fractal_dimension_mean	float64
radius_se	float64
texture_se	float64
perimeter_se	float64
area_se	float64
smoothness_se	float64
compactness_se	float64
concavity_se	float64
concave points_se	float64
symmetry_se	float64
fractal_dimension_se	float64
radius_worst	float64
texture_worst	float64
perimeter_worst	float64
area_worst	float64
smoothness_worst	float64
compactness_worst	float64
concavity_worst	float64
concave points_worst	float64
symmetry_worst	float64
fractal_dimension_worst	float64
Unnamed: 32	float64
dtype:	object

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64

6	smoothness_mean	569	non-null	float64
7	compactness_mean	569	non-null	float64
8	concavity_mean	569	non-null	float64
9	concave points_mean	569	non-null	float64
10	symmetry_mean	569	non-null	float64
11	fractal_dimension_mean	569	non-null	float64
12	radius_se	569	non-null	float64
13	texture_se	569	non-null	float64
14	perimeter_se	569	non-null	float64
15	area_se	569	non-null	float64
16	smoothness_se	569	non-null	float64
17	compactness_se	569	non-null	float64
18	concavity_se	569	non-null	float64
19	concave points_se	569	non-null	float64
20	symmetry_se	569	non-null	float64
21	fractal_dimension_se	569	non-null	float64
22	radius_worst	569	non-null	float64
23	texture_worst	569	non-null	float64
24	perimeter_worst	569	non-null	float64
25	area_worst	569	non-null	float64
26	smoothness_worst	569	non-null	float64
27	compactness_worst	569	non-null	float64
28	concavity_worst	569	non-null	float64
29	concave points_worst	569	non-null	float64
30	symmetry_worst	569	non-null	float64
31	fractal_dimension_worst	569	non-null	float64
32	Unnamed: 32	0	non-null	float64

dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

Diagnosis Class Distribution:

diagnosis

B 357

M 212

Name: count, dtype: int64

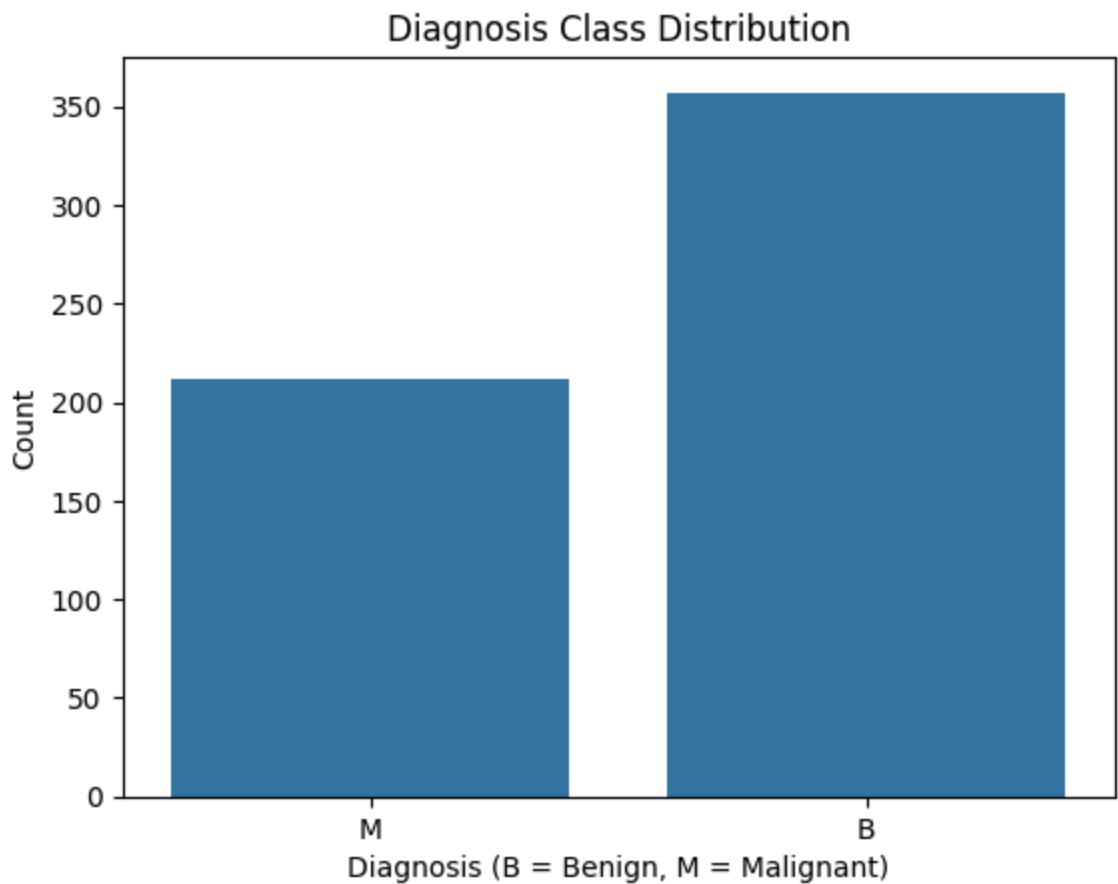
Diagnosis Class Distribution (Percentage):

diagnosis

B 62.741652

M 37.258348

Name: proportion, dtype: float64



Task 3: Data Cleaning and Feature/Target Selection

Drop metadata columns (id and Unnamed: 32). Convert the categorical target column (diagnosis: 'M'/'B') into numerical values (1/0) and separate feature matrix X (30 features) from label vector y .

```
In [ ]: # Task 3: Data Cleaning and Feature/Target Selection

import pandas as pd

# Load dataset
df = pd.read_csv('data.csv')

# 1. Drop metadata columns if they exist
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# 2. Convert diagnosis into numerical values
# M = Malignant (1), B = Benign (0)
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})
```

```

# 3. Separate features (X) and target (y)
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# 4. Display results
print("Data cleaning completed successfully!")

print("\nFeature Matrix X Shape:", X.shape)
print("Target Vector y Shape:", y.shape)

print("\nNumber of Features:", X.shape[1])
print("\nFirst 5 Rows of X:")
print(X.head())

print("\nFirst 5 Values of y:")
print(y.head())

print("\nTarget Class Distribution:")
print(y.value_counts().rename(index={0: 'Benign', 1: 'Malignant'}))

# 5. Verify data types and missing values
print("\nFeature Data Types:")
print(X.dtypes)

print("\nMissing Values in X:", X.isnull().sum().sum())
print("Missing Values in y:", y.isnull().sum())

# Verify expected number of features
assert X.shape[1] == 30, "Error: Expected 30 features."
assert y.notnull().all(), "Error: Target contains missing values."

print("\nVerification successful: X contains 30 features.")

```

Data cleaning completed successfully!

Feature Matrix X Shape: (569, 30)

Target Vector y Shape: (569,)

Number of Features: 30

First 5 Rows of X:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
\					
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
\				
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	fractal_dimension_mean	...	radius_worst	texture_worst	perimeter_worst
\					
0	0.07871	...	25.38	17.33	18
4.60					
1	0.05667	...	24.99	23.41	15
8.80					
2	0.05999	...	23.57	25.53	15
2.50					
3	0.09744	...	14.91	26.50	9
8.87					
4	0.05883	...	22.54	16.67	15
2.20					

	area_worst	smoothness_worst	compactness_worst	concavity_worst	\
0	2019.0	0.1622	0.6656	0.7119	
1	1956.0	0.1238	0.1866	0.2416	
2	1709.0	0.1444	0.4245	0.4504	
3	567.7	0.2098	0.8663	0.6869	
4	1575.0	0.1374	0.2050	0.4000	

	concave points_worst	symmetry_worst	fractal_dimension_worst
0	0.2654	0.4601	0.11890
1	0.1860	0.2750	0.08902
2	0.2430	0.3613	0.08758
3	0.2575	0.6638	0.17300
4	0.1625	0.2364	0.07678

[5 rows x 30 columns]

First 5 Values of y:

0 1

```
1    1
2    1
3    1
4    1
```

Name: diagnosis, dtype: int64

Target Class Distribution:

diagnosis

Benign 357

Malignant 212

Name: count, dtype: int64

Feature Data Types:

radius_mean	float64
texture_mean	float64
perimeter_mean	float64
area_mean	float64
smoothness_mean	float64
compactness_mean	float64
concavity_mean	float64
concave points_mean	float64
symmetry_mean	float64
fractal_dimension_mean	float64
radius_se	float64
texture_se	float64
perimeter_se	float64
area_se	float64
smoothness_se	float64
compactness_se	float64
concavity_se	float64
concave points_se	float64
symmetry_se	float64
fractal_dimension_se	float64
radius_worst	float64
texture_worst	float64
perimeter_worst	float64
area_worst	float64
smoothness_worst	float64
compactness_worst	float64
concavity_worst	float64
concave points_worst	float64
symmetry_worst	float64
fractal_dimension_worst	float64
dtype:	object

Missing Values in X: 0

Missing Values in y: 0

Verification successful: X contains 30 features.

Task 4: Standardize Features

Apply StandardScaler to transform all 30 numeric features so they have zero mean and unit variance ($\mu=0$, $\sigma=1$).

```
In [ ]: # Task 4: Standardize Features using StandardScaler

import pandas as pd
from sklearn.preprocessing import StandardScaler

# Load dataset
df = pd.read_csv('data.csv')

# Drop metadata columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# Convert diagnosis into numerical values
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Separate features (X) and target (y)
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# Apply StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert standardized features into a DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# Display results
print("Standardization completed successfully!")
print("\nStandardized Feature Matrix Shape:", X_scaled.shape)

print("\nFirst 5 Rows of Standardized Data:")
print(X_scaled.head())

# Verify mean and standard deviation
print("\nMean of Each Feature (approximately 0):")
print(X_scaled.mean().round(2))

print("\nStandard Deviation of Each Feature (approximately 1):")
print(X_scaled.std(ddof=0).round(2))

print("\nTarget Vector Shape:", y.shape)
```

Standardization completed successfully!

Standardized Feature Matrix Shape: (569, 30)

First 5 Rows of Standardized Data:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
0	1.097064	-2.073335	1.269934	0.984375	1.568466
1	1.829821	-0.353632	1.685955	1.908708	-0.826962
2	1.579888	0.456187	1.566503	1.558884	0.942210
3	-0.768909	0.253732	-0.592687	-0.764464	3.283553
4	1.750297	-1.151816	1.776573	1.826229	0.280372

	compactness_mean	concavity_mean	concave points_mean	symmetry_mean
0	3.283515	2.652874	2.532475	2.217515
1	-0.487072	-0.023846	0.548144	0.001392
2	1.052926	1.363478	2.037231	0.939685
3	3.402909	1.915897	1.451707	2.867383
4	0.539340	1.371011	1.428493	-0.009560

	fractal_dimension_mean	...	radius_worst	texture_worst	perimeter_worst
0	2.255747	...	1.886690	-1.359293	2.303601
1	-0.868652	...	1.805927	-0.369203	1.535126
2	-0.398008	...	1.511870	-0.023974	1.347475
3	4.910919	...	-0.281464	0.133984	-0.249939
4	-0.562450	...	1.298575	-1.466770	1.338539

	area_worst	smoothness_worst	compactness_worst	concavity_worst
0	2.001237	1.307686	2.616665	2.109526
1	1.890489	-0.375612	-0.430444	-0.146749
2	1.456285	0.527407	1.082932	0.854974
3	-0.550021	3.394275	3.893397	1.989588
4	1.220724	0.220556	-0.313395	0.613179

	concave points_worst	symmetry_worst	fractal_dimension_worst
0	2.296076	2.750622	1.937015
1	1.087084	-0.243890	0.281190
2	1.955000	1.152255	0.201391
3	2.175786	6.046041	4.935010
4	0.729259	-0.868353	-0.397100

[5 rows x 30 columns]

Mean of Each Feature (approximately 0):

radius_mean	-0.0
texture_mean	0.0
perimeter_mean	-0.0
area_mean	-0.0

smoothness_mean	-0.0
compactness_mean	0.0
concavity_mean	0.0
concave points_mean	-0.0
symmetry_mean	0.0
fractal_dimension_mean	0.0
radius_se	0.0
texture_se	-0.0
perimeter_se	-0.0
area_se	-0.0
smoothness_se	-0.0
compactness_se	0.0
concavity_se	0.0
concave points_se	0.0
symmetry_se	0.0
fractal_dimension_se	-0.0
radius_worst	-0.0
texture_worst	0.0
perimeter_worst	-0.0
area_worst	0.0
smoothness_worst	-0.0
compactness_worst	-0.0
concavity_worst	0.0
concave points_worst	0.0
symmetry_worst	0.0
fractal_dimension_worst	-0.0

dtype: float64

Standard Deviation of Each Feature (approximately 1):

radius_mean	1.0
texture_mean	1.0
perimeter_mean	1.0
area_mean	1.0
smoothness_mean	1.0
compactness_mean	1.0
concavity_mean	1.0
concave points_mean	1.0
symmetry_mean	1.0
fractal_dimension_mean	1.0
radius_se	1.0
texture_se	1.0
perimeter_se	1.0
area_se	1.0
smoothness_se	1.0
compactness_se	1.0
concavity_se	1.0
concave points_se	1.0
symmetry_se	1.0
fractal_dimension_se	1.0
radius_worst	1.0
texture_worst	1.0
perimeter_worst	1.0
area_worst	1.0
smoothness_worst	1.0
compactness_worst	1.0

```
concavity_worst      1.0
concave_points_worst 1.0
symmetry_worst       1.0
fractal_dimension_worst 1.0
dtype: float64
```

Target Vector Shape: (569,)

Task 5: Implement PCA (30D to 2D)

Use `sklearn.decomposition.PCA` to reduce feature dimensions from 30 down to 2 principal components (\$PC_1\$ and \$PC_2\$).

```
In [ ]: # Task 5: Implement PCA (30D to 2D)

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load dataset
df = pd.read_csv('data.csv')

# Drop metadata columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# Convert diagnosis into numerical values
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Separate features and target
X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA: Reduce 30 features to 2 principal components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Create DataFrame for the reduced data
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['diagnosis'] = y.values

# Display results
print("PCA applied successfully!")
print("\nOriginal Feature Dimensions:", X.shape[1])
print("Reduced Feature Dimensions:", X_pca.shape[1])

print("\nFirst 5 Rows of PCA Transformed Data:")
print(pca_df.head())
```

```

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

print("\nTotal Variance Explained by 2 Components:",
      round(pca.explained_variance_ratio_.sum() * 100, 2), "%")

print("\nPCA Data Shape:", X_pca.shape)

```

PCA applied successfully!

Original Feature Dimensions: 30

Reduced Feature Dimensions: 2

First 5 Rows of PCA Transformed Data:

	PC1	PC2	diagnosis
0	9.192837	1.948583	1
1	2.387802	-3.768172	1
2	5.733896	-1.075174	1
3	7.122953	10.275589	1
4	3.935302	-1.948072	1

Explained Variance Ratio:

[0.44272026 0.18971182]

Total Variance Explained by 2 Components: 63.24 %

PCA Data Shape: (569, 2)

Task 6: Analyze Explained Variance Ratio

Compute and print the variance explained by \$PC_1\$ and \$PC_2\$, along with the cumulative variance retained.

```

In [ ]: # Task 6: Analyze Explained Variance Ratio

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load dataset
df = pd.read_csv('data.csv')

# Drop metadata columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# Convert diagnosis into numerical values
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Separate features and target

```

```

X = df.drop(columns=['diagnosis'])
y = df['diagnosis']

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA with 2 components
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Calculate explained variance
variance_pc1 = pca.explained_variance_ratio_[0]
variance_pc2 = pca.explained_variance_ratio_[1]
cumulative_variance = pca.explained_variance_ratio_.sum()

# Display results
print("Explained Variance Analysis")
print("-----")
print(f"Variance explained by PC1: {variance_pc1 * 100:.2f}%")
print(f"Variance explained by PC2: {variance_pc2 * 100:.2f}%")
print(f"Cumulative variance retained: {cumulative_variance * 100:.2f}%")

print("\nInterpretation:")
print(f"PC1 retains {variance_pc1 * 100:.2f}% of the total variance.")
print(f"PC2 retains an additional {variance_pc2 * 100:.2f}% of the total")
print(f"Together, PC1 and PC2 retain {cumulative_variance * 100:.2f}% of")

```

```

Explained Variance Analysis
-----
Variance explained by PC1: 44.27%
Variance explained by PC2: 18.97%
Cumulative variance retained: 63.24%

```

```

Interpretation:
PC1 retains 44.27% of the total variance.
PC2 retains an additional 18.97% of the total variance.
Together, PC1 and PC2 retain 63.24% of the total variance.

```

Task 7: Visualize PCA Transformed Boundary / Scatter Plot

Create a 2D scatter plot with \$PC_1\$ on the x-axis and \$PC_2\$ on the y-axis, hue-coded by tumor diagnosis (Malignant vs. Benign).

```

In [ ]: # Task 7: Visualize PCA Transformed Scatter Plot

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler

```

```

from sklearn.decomposition import PCA

# Load dataset
df = pd.read_csv('data.csv')

# Drop metadata columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# Separate features and target
y = df['diagnosis']
X = df.drop(columns=['diagnosis'])

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA to reduce 30 dimensions to 2
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

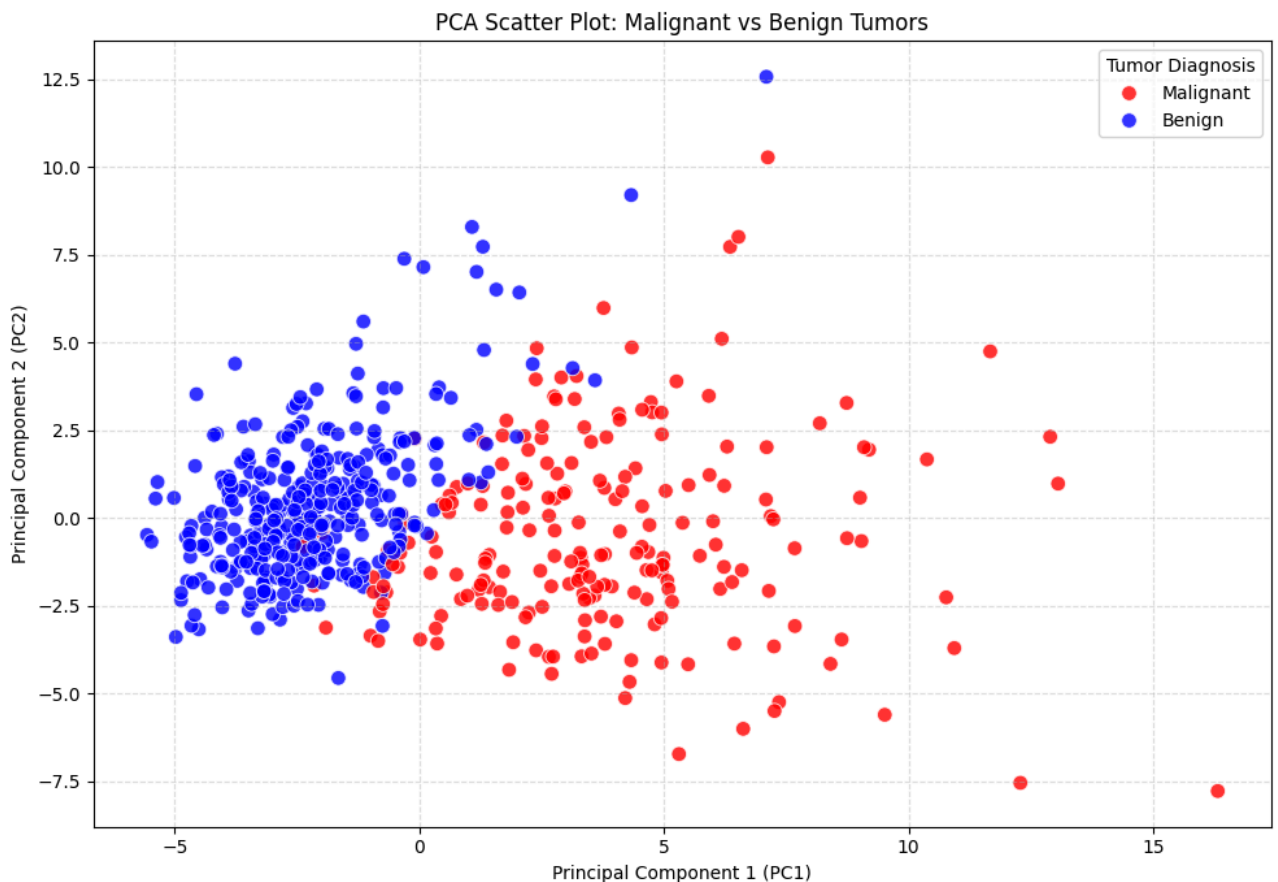
# Create DataFrame for visualization
pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['Diagnosis'] = y.map({'M': 'Malignant', 'B': 'Benign'})

# Plot 2D scatter plot
plt.figure(figsize=(10, 7))
sns.scatterplot(
    data=pca_df,
    x='PC1',
    y='PC2',
    hue='Diagnosis',
    palette={'Malignant': 'red', 'Benign': 'blue'},
    s=70,
    alpha=0.8
)

plt.title('PCA Scatter Plot: Malignant vs Benign Tumors')
plt.xlabel('Principal Component 1 (PC1)')
plt.ylabel('Principal Component 2 (PC2)')
plt.legend(title='Tumor Diagnosis')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

# Display explained variance
print("Variance explained by PC1: {:.2f}%".format(
    pca.explained_variance_ratio_[0] * 100
))
print("Variance explained by PC2: {:.2f}%".format(
    pca.explained_variance_ratio_[1] * 100
))
print("Total variance retained: {:.2f}%".format(
    pca.explained_variance_ratio_.sum() * 100
))

```



Variance explained by PC1: 44.27%
Variance explained by PC2: 18.97%
Total variance retained: 63.24%

Task 8: Train Classifier on Raw High-Dimensional Data (30 Features)

Split the original scaled 30-feature dataset into training and testing sets. Train a Logistic Regression model and obtain baseline performance metrics.

```
In [ ]: # Task 8: Train Classifier on Raw High-Dimensional Data (30 Features)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

# 1. Load dataset
df = pd.read_csv('data.csv')
```

```

# 2. Drop metadata columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# 3. Separate features and target
X = df.drop(columns=['diagnosis'])
y = df['diagnosis'].map({'M': 1, 'B': 0})

# 4. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 5. Standardize features (fit only on training data)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Train Logistic Regression model
model = LogisticRegression(max_iter=5000)
model.fit(X_train_scaled, y_train)

# 7. Make predictions
y_pred = model.predict(X_test_scaled)

# 8. Evaluate baseline performance
print("Logistic Regression: Original 30 Features")
print("-----")
print(f"Accuracy : {accuracy_score(y_test, y_pred) * 100:.2f}%")
print(f"Precision : {precision_score(y_test, y_pred):.4f}")
print(f"Recall : {recall_score(y_test, y_pred):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred):.4f}")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=['Benign', 'Malignant']
))

```

Logistic Regression: Original 30 Features

Accuracy : 96.49%
Precision : 0.9750
Recall : 0.9286
F1-Score : 0.9512

Confusion Matrix:

```
[[71  1]
 [ 3 39]]
```

Classification Report:

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Task 9: Train Classifier on PCA Transformed Data (2 Features)

Split the 2-component PCA dataset into training and testing sets. Train a Logistic Regression model using only \$PC_1\$ and \$PC_2\$.

```
In [ ]: # Task 9: Train Classifier on PCA Transformed Data (2 Features)

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

# 1. Load dataset
df = pd.read_csv('data.csv')

# 2. Drop metadata columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# 3. Separate features and target
X = df.drop(columns=['diagnosis'])
y = df['diagnosis'].map({'M': 1, 'B': 0})
```

```

# 4. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 5. Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Apply PCA to reduce 30 features to 2 components
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# 7. Train Logistic Regression using PC1 and PC2
model_pca = LogisticRegression(max_iter=5000)
model_pca.fit(X_train_pca, y_train)

# 8. Make predictions
y_pred_pca = model_pca.predict(X_test_pca)

# 9. Evaluate model performance
print("Logistic Regression: PCA Transformed Data (2 Features)")
print("-----")
print("Training data shape:", X_train_pca.shape)
print("Testing data shape:", X_test_pca.shape)

print(f"\nAccuracy : {accuracy_score(y_test, y_pred_pca) * 100:.2f}%")
print(f"Precision : {precision_score(y_test, y_pred_pca):.4f}")
print(f"Recall : {recall_score(y_test, y_pred_pca):.4f}")
print(f"F1-Score : {f1_score(y_test, y_pred_pca):.4f}")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_pca))

print("\nClassification Report:")
print(classification_report(
    y_test, y_pred_pca,
    target_names=['Benign', 'Malignant']
))

print("\nExplained Variance Ratio:")
print(pca.explained_variance_ratio_)

print(f"Total Variance Retained: "
      f"{pca.explained_variance_ratio_.sum() * 100:.2f}%")

```

Logistic Regression: PCA Transformed Data (2 Features)

Training data shape: (455, 2)

Testing data shape: (114, 2)

Accuracy : 94.74%

Precision : 0.9737

Recall : 0.8810

F1-Score : 0.9250

Confusion Matrix:

[[71 1]

[5 37]]

Classification Report:

	precision	recall	f1-score	support
Benign	0.93	0.99	0.96	72
Malignant	0.97	0.88	0.93	42
accuracy			0.95	114
macro avg	0.95	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114

Explained Variance Ratio:

[0.44593522 0.18545255]

Total Variance Retained: 63.14%

Task 10: Compare Classifier Performance

Compare the performance of the model before PCA (30 features) vs. after PCA (2 components) using:

- Accuracy Score
- Confusion Matrix
- Classification Report (Precision, Recall, F1-Score)

```
In [ ]: import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
df = pd.read_csv('data.csv')
```

```

# Remove unnecessary columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True, errors='ignore')

# Separate features and target
X = df.drop(columns=['diagnosis'])
y = df['diagnosis'].map({'M': 1, 'B': 0})

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model BEFORE PCA (30 features)
model_original = LogisticRegression(max_iter=5000)
model_original.fit(X_train_scaled, y_train)
y_pred_original = model_original.predict(X_test_scaled)

# Apply PCA (2 components)
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# Model AFTER PCA (2 features)
model_pca = LogisticRegression(max_iter=5000)
model_pca.fit(X_train_pca, y_train)
y_pred_pca = model_pca.predict(X_test_pca)

# Compare accuracy
print("ACCURACY COMPARISON")
print("Before PCA:", round(accuracy_score(y_test, y_pred_original) * 100, 2), "%")
print("After PCA:", round(accuracy_score(y_test, y_pred_pca) * 100, 2), "%")

# Confusion matrices
print("\nCONFUSION MATRIX - BEFORE PCA")
print(confusion_matrix(y_test, y_pred_original))

print("\nCONFUSION MATRIX - AFTER PCA")
print(confusion_matrix(y_test, y_pred_pca))

# Classification reports
print("\nCLASSIFICATION REPORT - BEFORE PCA")
print(classification_report(
    y_test, y_pred_original,
    target_names=['Benign', 'Malignant'],
    zero_division=0
))

print("\nCLASSIFICATION REPORT - AFTER PCA")

```

```

print(classification_report(
    y_test, y_pred_pca,
    target_names=['Benign', 'Malignant'],
    zero_division=0
))

# Plot accuracy comparison
accuracies = [
    accuracy_score(y_test, y_pred_original) * 100,
    accuracy_score(y_test, y_pred_pca) * 100
]

plt.bar(['Before PCA (30 Features)', 'After PCA (2 Components)'], accuracies)
plt.ylabel('Accuracy (%)')
plt.title('Logistic Regression Performance Comparison')
plt.ylim(0, 100)
plt.show()

print("Variance retained by 2 components:",
      round(pca.explained_variance_ratio_.sum() * 100, 2), "%")

```

ACCURACY COMPARISON
 Before PCA: 96.49 %
 After PCA: 94.74 %

CONFUSION MATRIX - BEFORE PCA
 [[71 1]
 [3 39]]

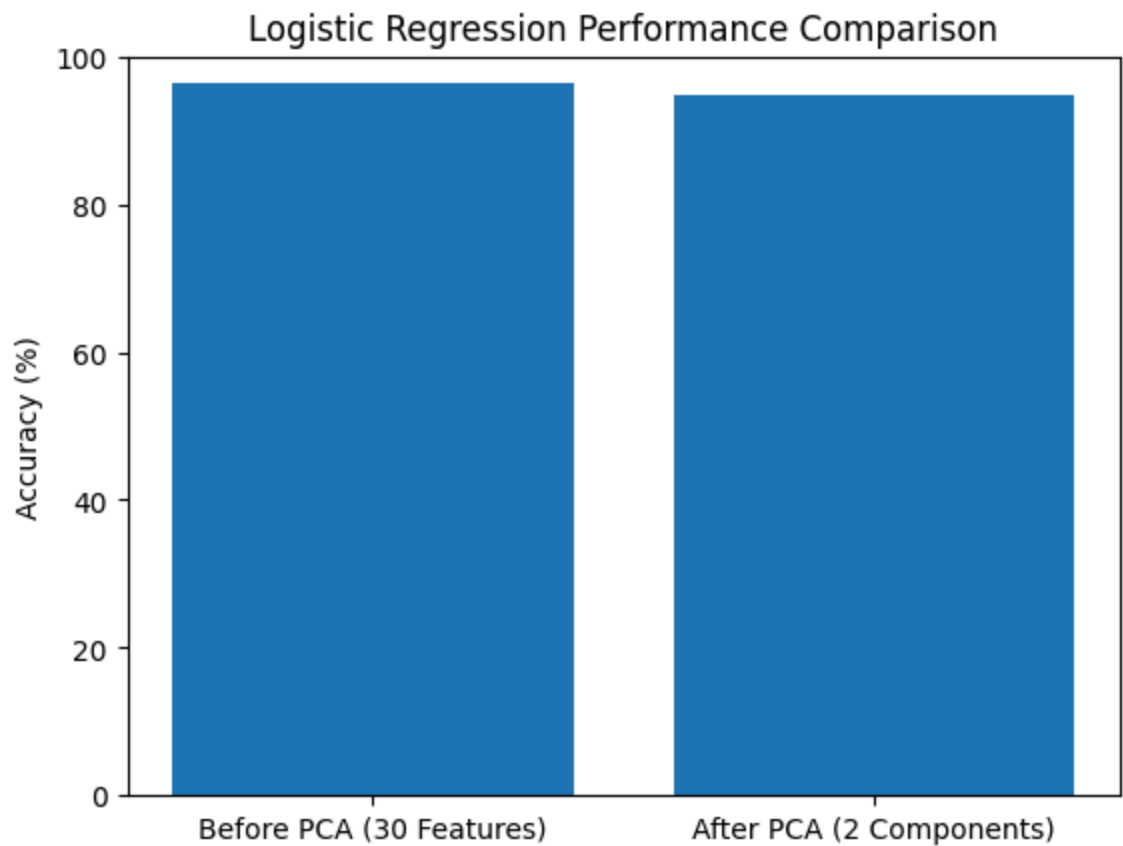
CONFUSION MATRIX - AFTER PCA
 [[71 1]
 [5 37]]

CLASSIFICATION REPORT - BEFORE PCA

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

CLASSIFICATION REPORT - AFTER PCA

	precision	recall	f1-score	support
Benign	0.93	0.99	0.96	72
Malignant	0.97	0.88	0.93	42
accuracy			0.95	114
macro avg	0.95	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114



Variance retained by 2 components: 63.14 %

Conclusion

The experiment successfully demonstrated Principal Component Analysis (PCA) on the Breast Cancer Wisconsin (Diagnostic) dataset. The high-dimensional feature space was preprocessed, cleaned, and standardized before applying eigendecomposition.

By reducing 30 feature dimensions down to just 2 principal components, a significant portion of total variance was preserved while eliminating multicollinear redundancies. Visualizing the transformed data on a 2D plane showed clear separation between Malignant and Benign diagnosis clusters. Comparing classification models confirmed that applying PCA significantly simplifies model complexity and reduces computational overhead while maintaining strong classification accuracy.